

中国科学技术大学

本科毕业论文



面向 GPU 数值分解的单层 k 叉增量 重排序算法

作者姓名：	计勃帆
学 号：	PB22010389
专 业：	信息与计算科学
导 师：	陈仁杰 教授
完成时间：	2026 年 5 月 2 日

中国科学技术大学本科毕业论文学术诚信承诺书

本人郑重承诺：所呈交的本科毕业论文，是本人在导师指导下进行研究工作所取得的成果。除已特别加以标注和致谢的地方外，论文中不包含任何他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的贡献均已在论文中作了明确的说明。如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文(设计)等违纪行为。

作者签名：_____

签字日期：_____

摘要

稀疏 Cholesky 重分解是在原稀疏对称正定矩阵发生局部修改时，复用已有分解结果以高效获得新分解的重要算法，在网格参数化、物理仿真和几何处理等计算机图形学应用中具有重要意义。稀疏 Cholesky 重分解通常包含符号分析与数值计算两个阶段，其中符号分析包括填充减少排序、符号分解、消元树构建与超节点识别等步骤，直接决定后续数值分解的稀疏结构、依赖关系和并行调度方式。然而，现有方法大多面向 CPU 架构设计，在大规模问题中容易受到并行度不足、访存不规则和内存带宽限制的影响，难以充分发挥现代 GPU 的计算潜力。因此，本文拟面向 GPU 并行架构，研究并实现一种适配后续 GPU 数值分解的稀疏 Cholesky 重分解算法。

由于填充减少排序问题本身为 NP 困难，现有工作通常采用启发式策略，并在 CPU 上取得了较成熟的效果。本文实现的填充减少排序算法仍在 CPU 上完成，其目的并非直接将排序过程本身迁移到 GPU，而是通过重新组织稀疏矩阵对应图的划分结构，构造更适合后续 GPU 并行数值分解的分解树。传统层次二叉树分解虽然能够通过递归划分逐步构建较好的分解结构，但其分解过程具有较强的逐层依赖性：每一层划分往往依赖上一层结果，树的深度较大，节点规模变化不均，且不同层之间难以同时展开计算。这种逐层递归结构会在后续 GPU 数值分解阶段引入较多同步、调度和不规则访存开销，导致线程占用率不足，因而不利于大规模并行执行。相比之下，单层 k 叉树分解通过一次性构造多个规模相对均衡的子区域，使分解树中同层节点之间的依赖关系显著降低，不同子树可以更独立地进行局部符号分析、局部重分解和数值更新，从而为 GPU 提供更充足且更规则的并行任务粒度。基于这一特点，本文采用 CPU 端的 k 叉树填充减少排序构建适合 GPU 并行计算的分解树结构，并在此基础上设计后续符号分析与数值重分解流程。实验结果表明，在保证分解质量的前提下，本文方法显著提升了 10^5 量级大规模网格在局部修改场景下的稀疏 Cholesky 重分解效率，为大规模图形仿真和几何处理提供高性能基础算法支持。

关键词：稀疏 Cholesky 重分解；填充减少排序；k 叉树，GPU 并行计算

ABSTRACT

Sparse Cholesky refactorization is an important technique for efficiently updating the factorization of a sparse symmetric positive definite matrix after local modifications. It reuses information from the previous factorization and plays a key role in computer graphics applications such as mesh parameterization, physical simulation, and geometric processing. Sparse Cholesky refactorization usually consists of two stages: symbolic analysis and numerical computation. Symbolic analysis includes fill-reducing ordering, symbolic factorization, elimination tree construction, and supernode detection, which determine the sparsity pattern, dependency structure, and parallel scheduling strategy of the subsequent numerical factorization. However, most existing methods are designed for CPU architectures. When applied to large-scale problems, they are often limited by insufficient parallelism, irregular memory access, and memory bandwidth, making it difficult to fully exploit the computational capability of modern GPUs. Therefore, this thesis studies and implements a sparse Cholesky refactorization algorithm adapted to subsequent GPU-based numerical factorization.

Since the fill-reducing ordering problem is NP-hard, existing methods usually rely on heuristic strategies and have achieved mature performance on CPUs. The fill-reducing ordering algorithm implemented in this thesis is still performed on the CPU. Its purpose is not to migrate the ordering process itself to the GPU, but to reorganize the graph partitioning structure of the sparse matrix and construct a decomposition tree that is more suitable for subsequent GPU-parallel numerical factorization. Traditional hierarchical binary tree decomposition can gradually construct a high-quality decomposition structure through recursive partitioning, but its process has strong level-by-level dependencies: each level often depends on the result of the previous level, the tree depth is large, node sizes are uneven, and different levels are difficult to process simultaneously. Such a recursive structure introduces considerable synchronization, scheduling, and irregular memory access overhead in the subsequent GPU numerical factorization stage, leading to insufficient thread occupancy and poor suitability for large-scale parallel execution. In contrast, single-level k -ary tree decomposition constructs multiple

relatively balanced subregions at once, significantly reducing the dependencies among nodes at the same level. Different subtrees can then perform local symbolic analysis, local refactorization, and numerical updates more independently, providing the GPU with more abundant and regular parallel task granularity. Based on this observation, this thesis adopts a CPU-side k -ary tree fill-reducing ordering method to construct a GPU-friendly decomposition tree, and further designs the subsequent symbolic analysis and numerical refactorization workflow. Experimental results show that, while maintaining factorization quality, the proposed method significantly improves the sparse Cholesky refactorization efficiency for large-scale meshes of order 10^5 under local modifications, providing a high-performance fundamental algorithm for large-scale graphics simulation and geometric processing.

Key Words: Sparse Cholesky refactorization; Fill-reducing ordering; k -ary tree; GPU parallel computing

目 录

第一章 研究背景与问题定义	4
第一节 研究背景与研究动机	4
第二节 问题定义与技术挑战	5
第三节 研究目标与技术路线	7
第四节 论文组织结构	8
第二章 相关研究	10
第一节 近似最小度排序算法	10
一、最小度启发式与填充元控制	10
二、商图模型与近似度数	11
三、算法流程与适用特点	11
第二节 METIS 多路图划分	12
一、多级图划分框架	12
二、多路划分的算法特征	13
第三节 嵌套剖分排序算法	13
一、顶点分割与分隔集思想	13
二、递归二分排序流程	14
三、层次二叉结构的局限性	14
第四节 Parth 稀疏 Cholesky 重分解方法	15
第五节 消元树	16
一、消元树的定义	16
二、数据依赖与并行性分析	16
三、符号分解与内存预分配	16
四、消元树与填充减少排序的关系	17
第三章 面向 GPU 数值分解的单层 k 叉增量重排序算法	18
第一节 单层 k 叉分解树的结构	18
第二节 初始化阶段	20

第三节 增量二次重排序	22
一、受影响子图分类	22
二、局部划分与置换更新	25
第四节 算法特性分析	28
第五节 应用场景	28
第四章 实验评估	30
第一节 实验环境与测试数据	30
第二节 运行时间实验	30
一、cathead 网格运行时间分析	31
二、ra 网格运行时间分析	31
三、igea 网格运行时间分析	32
第三节 置换结构可视化实验	32
第四节 本章小结	35
第五章 总结与展望	37
第一节 工作总结	37
第二节 未来展望	38
参考文献	39

插图和附表清单

图 3.1	多层二叉分解树与单层 k 叉分解树结构对比	19
图 3.2	初始化阶段的单层 k 叉结构示例	20
图 3.3	增量二次重排序中的子图分类示例	23
图 3.4	增量二次重排序中的局部划分与置换更新示例	26
图 4.1	cathead 网格运行时间	31
图 4.2	ra 网格运行时间	31
图 4.3	igea 网格运行时间	32
图 4.4	igea 网格置换结构可视化	33
图 4.5	igea 网格置换后的消元树可视化	34
表 4.1	不同置换结构对应的消元树高度	34

第一章 研究背景与问题定义

第一节 研究背景与研究动机

在计算机图形学、科学计算与工程仿真等领域，大规模稀疏线性方程组的高效求解是许多核心算法的关键计算环节。设 A 表示系统矩阵， x 表示待求解变量， b 表示右端项，则相关问题通常可写为

$$Ax = b. \quad (1.1)$$

在网格参数化、三维几何处理以及基于有限元方法的物理仿真等应用中，系统矩阵 A 往往具有高度稀疏性，并且在许多情形下是对称正定矩阵。为了保证数值稳定性与求解精度，直接法中的稀疏 Cholesky 分解被广泛用于此类问题。相关研究表明，在交互式网格参数化、子网格分解复用以及动态物理仿真中，稀疏 Cholesky 分解及其更新技术是提升整体求解效率的重要基础^[1-3]。然而，随着三维模型分辨率不断提高以及物理仿真场景日益复杂，系统矩阵规模可能达到十万、百万甚至更高量级。若每次求解都从头执行完整的符号分析与数值分解，将产生较高的时间与内存开销。

在交互式图形应用和时间步进仿真过程中，系统矩阵通常不会在每一步发生全局变化。更常见的情形是，矩阵仅因局部几何编辑、局部拓扑修改、局部碰撞响应或材料参数变化而发生局部更新。也就是说，原矩阵 A 只有少量非零元素的数值或结构发生改变。在此类动态场景中，如果每次更新后都将矩阵视为一个全新的矩阵并重新执行完整 Cholesky 分解，会造成大量重复计算，也难以满足交互式应用对计算效率的要求。

稀疏 Cholesky 重分解技术正是针对上述问题提出的。其基本思想是：在矩阵发生局部修改后，尽可能复用已有的符号结构、消元依赖关系和数值分解结果，仅对受影响的局部区域进行更新，从而降低连续求解过程中的重复计算成本。矩阵分解更新和重分解问题已有较长的研究历史，早期工作主要关注矩阵因子在低秩修改下的更新技术^[4-5]；随后，稀疏 Cholesky 分解的修改、动态超节点维护以及稀疏结构更新得到了进一步研究^[6-7]。这些技术对于网格参数化、几何优化、物理仿真和动态图形处理等应用具有重要意义。

现有稀疏 Cholesky 分解与重分解方法大多面向 CPU 架构设计。CPU 具有较

强的复杂控制流处理能力, 适合执行递归式图算法、启发式排序以及不规则数据结构遍历。然而, 在大规模问题中, CPU 的并行核心数和内存带宽有限, 难以充分满足现代图形仿真和几何处理对高吞吐计算的需求。相比之下, GPU 具有大量计算核心和较高显存带宽, 更适合执行大规模、规则化和高并行度的数值计算任务。近年来, GPU 稀疏 Cholesky 分解以及 GPU 上的稀疏矩阵重排序也逐渐成为相关研究关注的问题^[8-9]。因此, 如何构造适合 GPU 后续数值分解的稀疏矩阵分解结构, 是提升大规模稀疏 Cholesky 重分解效率的重要问题。

需要指出的是, 本文并不将填充减少排序过程本身完全迁移到 GPU 上。相反, 本文实现的填充减少排序算法仍在 CPU 端执行, 其目标是通过重新设计图划分与分解树构造方式, 生成更适合后续 GPU 并行数值分解的稀疏结构。换言之, 本文关注的核心并不是简单地将传统 CPU 排序算法移植到 GPU, 而是通过 CPU 端的结构预处理, 为 GPU 端数值重分解提供更规则、更均衡且并行度更高的任务组织形式。

第二节 问题定义与技术挑战

设 $A \in \mathbb{R}^{n \times n}$ 表示一个大规模稀疏对称正定矩阵, 其中 n 表示矩阵维度。稀疏 Cholesky 分解通常可写为

$$PAP^T = LL^T, \quad (1.2)$$

其中, P 表示用于降低填充元数量的置换矩阵, P^T 表示 P 的转置矩阵, L 表示对角元为正的下三角 Cholesky 因子。置换矩阵 P 的选择会直接影响因子 L 的非零结构、填充元数量、消元树形态以及后续数值分解的并行调度方式。计算最小填充排序本身是 NP 完全问题, 因此实际求解器通常依赖启发式排序或图划分方法获得近似较优的置换^[10]。

当仿真状态或几何模型发生局部变化时, 原矩阵 A 更新为

$$\tilde{A} = A + \Delta A, \quad (1.3)$$

其中, $\tilde{A}$ 表示更新后的矩阵, ΔA 表示由局部修改引起的矩阵变化。通常情况下, ΔA 的非零范围远小于 A 的整体规模。重分解问题的目标是在已有分解结构的基础上, 高效计算更新后的分解形式

$$\tilde{P}\tilde{A}\tilde{P}^T = \tilde{L}\tilde{L}^T, \quad (1.4)$$

其中, $\tilde{P}$ 表示更新后的置换矩阵, $\tilde{L}$ 表示更新后的 Cholesky 因子。

完整的稀疏 Cholesky 重分解流程通常包括符号分析和数值计算两个阶段。符号分析包括填充减少排序、符号分解、消元树构建和超节点识别等步骤，其作用是确定分解因子的非零结构以及计算依赖关系。数值计算阶段则在符号结构确定后执行具体的矩阵分解更新。对于 GPU 而言，数值计算阶段可以借助高度优化的稠密线性代数算子获得较高效率，而符号结构是否具有足够的并行性、规则性和负载均衡性，会直接影响后续 GPU 数值分解的实际性能^[8]。

针对这一问题，本文面临的主要技术挑战如下。

- **传统启发式排序算法的序列依赖较强：**经典填充减少排序算法，如近似最小度排序算法（Approximate Minimum Degree, AMD），通常采用局部贪心策略，在每一步选择当前代价较小的节点进行模拟消元，并动态更新相关邻接结构。AMD 算法能够在实际稀疏矩阵中取得较好的填充减少效果，但其消元选择和度数更新过程仍然具有明显的前后依赖^[11]。因此，该类方法虽然在 CPU 上较为成熟，但难以直接为 GPU 数值分解构造出高并行度、低同步开销的任务结构。
- **层次二叉树分解不利于 GPU 任务展开：**基于嵌套剖分排序算法（Nested Dissection, ND）的传统图划分方法通常采用递归式层次二叉树分解。该方法能够逐层构造分隔集合，并在一定程度上降低分解过程中的填充元数量。然而，层次二叉树结构具有较深的递归层级和较强的逐层依赖关系。上层划分完成之前，下层划分通常难以独立展开；不同层之间的任务规模差异较大，同层节点数量也可能不足以提供充足并行任务。这种结构在 CPU 上可以通过递归调度自然处理，但在 GPU 后续数值分解阶段容易引入频繁同步、任务粒度不均和控制流分化等问题。
- **不规则图结构导致并行负载难以均衡：**稀疏矩阵对应的图通常具有不规则拓扑结构，节点度数分布不均，局部连通关系复杂。无论是图划分、消元树构建，还是受影响区域识别，都可能涉及大量间接寻址和不规则访存。若分解树结构缺乏良好的均衡性，不同计算任务的规模差异会进一步放大 GPU 上的负载不均衡问题，降低后续数值分解阶段的硬件利用率。
- **局部修改下结构复用与局部更新需要兼顾：**在重分解场景下，矩阵修改通常具有局部性。若每次局部修改后都执行全局重新排序，不仅会引入较高的符号分析开销，还可能导致前后两次分解结构差异过大，从而削弱已有数值因子的复用价值。现有动态稀疏结构复用方法已经表明，合理复用历史排序和符号结构能够显著降低连续求解中的重复计算成本^[12]。因此，重

分解算法需要在保持整体分解质量的同时，将局部修改的影响范围尽可能限制在少数子结构内，并避免对全局分解树进行不必要的重构。

因此，本文关注的核心问题可以概括为：如何在 CPU 端构造一种适合 GPU 后续数值分解的填充减少排序与分解树结构，使其在保持较好分解质量的同时，降低层次递归依赖，增强节点之间的独立性，并为局部修改场景下的高效重分解提供结构基础。

第三节 研究目标与技术路线

针对上述问题，本文的总体目标是设计并实现一种面向 GPU 数值重分解的稀疏 Cholesky 重分解框架。本文的重点在于 CPU 端填充减少排序与分解树结构的重新设计，即通过构造更适合 GPU 并行执行的分解树，为后续数值分解阶段提供更充分、更均衡且依赖更弱的并行任务。

传统 ND 方法通常采用递归二分策略构造层次二叉树。虽然这种方法能够获得较好的填充减少效果，但其树结构较深，逐层依赖明显，并且高层节点数量较少，难以在后续计算中提供足够的并行任务。针对这一不足，本文采用单层 k 叉树分解思路，将全局稀疏矩阵对应的图一次性划分为 k 个规模相对均衡的子区域，其中 k 表示划分得到的子区域数量。与深层二叉树结构相比，单层 k 叉树具有更浅的树深度和更多的同层独立节点，有利于降低节点之间的依赖关系，并为 GPU 数值分解阶段提供更大的并行任务粒度。

围绕这一思路，本文的具体研究目标包括以下几个方面。

- **构造面向 GPU 数值分解的 CPU 端填充减少排序方法：**本文在 CPU 上实现基于 k -way 图划分的填充减少排序方法。该方法的目标不是提升排序阶段本身的 GPU 并行效率，而是通过生成更规则、更均衡的分解树结构，使后续 GPU 数值分解阶段能够获得更高的并行度和更低的同步开销。
- **以单层 k 叉树替代深层层次二叉树结构：**本文通过单层多路划分直接构造多个相对独立的子图，减少传统递归二叉分解带来的逐层依赖和任务粒度不均问题。该结构能够提高同层子任务之间的独立性，使局部符号分析、局部重分解和数值更新更容易并行展开。
- **设计局部修改场景下的增量重排序机制：**当稀疏矩阵发生局部结构修改时，本文通过识别受影响的子图范围，将重排序和重分解操作尽量限制在相关分支内部，避免不必要的全局重新排序。该机制有助于复用已有分解结构，

并降低局部修改后的重分解时间。

- **评估分解质量与重分解效率之间的权衡关系：**本文通过实验分析不同划分参数、不同局部修改规模和不同网格模型下算法的运行时间与结构特征，验证所提出方法在保持分解质量的前提下，对 10^5 量级大规模网格局部修改场景中重分解效率的提升效果。

通过上述设计，本文希望在符号结构层面为 GPU 数值分解提供更合适的任务组织方式，从而缓解传统分解树深度较大、逐层依赖较强和任务负载不均衡等问题，为大规模图形仿真与几何处理中的稀疏线性系统连续求解提供更高效的基础算法。

第四节 论文组织结构

本文后续章节安排如下。

第二章相关研究：本章系统回顾稀疏 Cholesky 分解、填充减少排序、图划分和重分解相关研究。首先介绍 AMD 及其局部贪心排序思想；随后讨论 METIS 中的 k-way 图划分方法，说明其在构造均衡子图和组织并行任务方面的作用；接着介绍 ND 排序算法，重点说明顶点分割与递归二分结构在稀疏矩阵排序中的意义；然后介绍 Parth 稀疏 Cholesky 重分解方法，分析其对动态稀疏结构的增量复用思想及其层次二叉结构在 GPU 数值分解场景下的局限；最后介绍消元树这一稀疏矩阵符号分析中的核心数据结构。

第三章面向 GPU 数值分解的单层 k 叉增量重排序算法：本章详细介绍本文提出的 CPU 端单层 k 叉增量重排序算法。首先从 Parth 方法的层次二叉结构出发，分析其在 GPU 后续数值分解中的任务展开、负载均衡和同步开销问题；随后说明本文采用单层 k 叉分解树的动机和数据结构设计；接着介绍初始化阶段中基于 METIS k-way 划分和 AMD 的初始排序流程；最后重点说明新增边场景下的受影响子图分类、局部划分更新与局部置换更新机制，并进一步总结算法特性和典型应用场景。

第四章实验评估：本章对本文方法进行实验评估。首先说明实验环境、依赖库配置以及测试网格模型；随后通过改变划分区域数 k 、局部新增边数量和网格规模，比较初始化阶段、首次 AMD 重排序阶段以及二次重排序阶段的运行时间；接着结合不同规模网格的实验结果分别分析算法适用范围和参数权衡；最后通过置换矩阵可视化结果，分析本文方法在保持分解结构质量、形成分块模式以及

支持局部重分解方面的效果。

第五章总结与展望：本章总结全文工作，概括本文在 CPU 端构造 GPU 友好分解树、单层 k 叉划分以及局部增量重排序方面的主要贡献。同时，讨论当前方法的局限性，并展望未来工作，包括进一步优化 GPU 数值分解阶段的任务调度、加强符号分析与数值计算之间的协同设计，以及支持更复杂的动态拓扑修改场景。

第二章 相关研究

稀疏 Cholesky 分解的计算效率在很大程度上取决于矩阵非零结构的组织方式。对于大规模稀疏对称正定矩阵而言，填充减少排序、图划分、消元树构建以及局部重分解机制共同决定了分解因子的稀疏性、计算依赖关系和并行调度效率。因此，在研究面向 GPU 数值分解的重分解算法之前，有必要首先回顾若干具有代表性的基础算法与相关工作。

本章首先介绍 AMD，分析其通过局部贪心策略减少填充元的基本思想；随后介绍 METIS 图划分库中的 k -way 划分方法，说明其在构造均衡子图和组织并行任务方面的作用；接着讨论 ND，重点说明顶点分割与递归二分结构在稀疏矩阵排序中的意义；然后介绍 Parth 稀疏 Cholesky 重分解方法，分析其对动态稀疏结构的增量更新思想；最后介绍消元树这一稀疏矩阵符号分析中的核心数据结构，为后续章节的算法设计奠定基础。

第一节 近似最小度排序算法

在稀疏矩阵填充减少排序中，最小度算法是最经典的局部贪心启发式方法之一。AMD 是在最小度算法基础上发展而来的高效排序算法，其核心目标是在避免精确维护复杂图结构的前提下，近似模拟最小度消元过程，从而获得较好的填充减少效果和较低的计算开销^[1]。

一、最小度启发式与填充元控制

对于一个稀疏对称正定矩阵 A ，其非零结构可以等价表示为一个无向图 $G = (V, E)$ ，其中 V 表示图中的顶点集合， E 表示图中的边集合。矩阵中的每一个变量对应图中的一个顶点；若矩阵中第 i 行第 j 列存在非零元，则图中对应顶点之间存在一条边。

在图模型下，消去一个顶点 v 等价于将该顶点从图中移除，并将它所有尚未消去的邻接顶点两两相连，形成一个完全子图。消元过程中新增的边对应于矩阵分解因子中产生的填充元。最小度算法的基本策略是在每一步选择当前图中度数最小的顶点进行消元，以局部贪心的方式尽可能减少当前步骤引入的填充元数量。

然而，随着消元过程的推进，图中会不断产生新的团结构。如果显式添加所有填充边并精确维护每个未消元顶点的当前度数，将导致较高的时间开销和内存开销。对于大规模稀疏矩阵而言，这种精确维护方式往往难以承受。并且，从理论上看，最小填充排序问题本身具有较高计算复杂性，已被证明为 NP 完全问题^[10]。因此，如何在保持较好排序质量的同时降低度数更新成本，成为最小度类算法改进的关键问题。

二、商图模型与近似度数

AMD 的主要思想是放弃对真实度数的精确计算，转而使用一个计算代价更低的近似度数作为贪心选择依据。该方法由 Amestoy、Davis 和 Duff 提出，其关键创新在于引入商图模型，用隐式方式表示消元过程中形成的团结构^[11]。

在商图模型中，图中的对象被分为两类。第一类是尚未消元的原始顶点，称为变量；第二类是已经消元的顶点及其诱导形成的团结构，称为元素。当一个顶点被消去后，算法不再显式添加由其邻居形成的所有填充边，而是将这一团结构记录为一个元素。之后，某个未消元变量的近似度数可以通过统计与其相邻的变量和元素所覆盖的变量集合来估计。

这种表示方式避免了在原图中显式维护大量新增填充边，从而显著降低了消元模拟过程中的存储成本和更新成本。虽然近似度数并不总是等于真实度数，但在实际应用中通常能够给出较好的排序结果，因此 AMD 被广泛用于稀疏矩阵直接求解器的预处理阶段。

三、算法流程与适用特点

AMD 的典型执行流程可以概括如下：

1. 根据稀疏矩阵 A 的非零结构构造无向图 $G = (V, E)$ ，并初始化每个顶点的近似度数；
2. 从尚未消元的顶点中选择近似度数最小的顶点作为当前消元顶点；
3. 将当前消元顶点及其引入的团结构合并为新的元素，并更新相关变量与元素之间的邻接关系；
4. 重新估计受影响变量的近似度数，并继续选择下一个消元顶点；
5. 重复上述过程，直到所有顶点均被消元，最终得到一个填充减少排序。

AMD 具有实现成熟、排序质量较好、对一般稀疏矩阵适应性较强等优点。然而，该算法本质上仍然是一种逐步消元的局部贪心方法。每一步的选择都依赖

于前一步消元后更新得到的近似图结构，因此存在较强的序列依赖。对于 CPU 而言，这种复杂控制流和动态数据结构尚可高效处理；但对于依赖大规模并行任务展开的 GPU 而言，该类算法难以直接提供充分的并行粒度。因此，在面向 GPU 数值分解的场景下，仅依赖 AMD 排序并不能充分发挥硬件并行能力。

第二节 METIS 多路图划分

METIS 是一种广泛使用的图划分与稀疏矩阵排序软件库。其核心思想是采用多级图划分范式，在降低计算复杂度的同时获得较高质量的图划分结果。与逐点贪心消元方法不同，图划分方法从全局结构出发，将图划分为若干规模相对均衡、边界连接较少的子图，从而为分治求解和并行计算提供结构基础^[13-14]。

本文主要关注 METIS 中的 k -way 划分方法。该方法将原始图直接划分为 k 个子图，其中 k 表示划分得到的子图数量。与反复执行二路划分相比， k -way 划分能够在更全局的视角下同时协调多个分区之间的边界关系，因此更适合构造单层 k 叉树结构。

一、多级图划分框架

METIS 的多级图划分通常包含三个阶段：粗化阶段、初始划分阶段以及反粗化优化阶段^[13-14]。

在粗化阶段，算法通过顶点匹配或聚合操作，将原始图逐步压缩为一系列规模更小的粗图。粗化的目标是在尽量保留原图整体结构特征的同时降低图规模，从而为后续划分降低计算成本。

在初始划分阶段，算法在最粗层图上计算一个初始的 k 路划分。由于此时图的规模已经显著缩小，可以使用相对复杂的启发式策略获得较好的初始划分结果。

在反粗化与优化阶段，算法将粗图上的划分结果逐级映射回更细层图，并在每一层上执行局部优化。常见的局部优化方法包括 KL 和 FM。这些方法通过移动边界顶点来减小割边数量或改善分区平衡性。

二、多路划分的算法特征

在 k 路划分问题中，目标是将图 $G = (V, E)$ 的顶点集合 V 划分为 k 个互不相交的子集：

$$V = V_1 \cup V_2 \cup \dots \cup V_k, \quad (2.1)$$

其中， V_i 表示第 i 个子图的顶点集合，并且对于任意 $i \neq j$ ，有

$$V_i \cap V_j = \emptyset. \quad (2.2)$$

划分过程通常要求各个子图规模尽可能均衡，同时尽量减少不同子图之间的连接边数量。跨越不同子图的边称为割边，其数量反映了分区之间的耦合程度。

直接 k -way 划分的优势在于，算法能够同时考虑 k 个分区之间的全局关系，而不是通过多次独立二分逐步生成结果。递归二分可能在早期划分中作出局部最优但全局不佳的决策，导致后续分区质量下降。相比之下， k -way 划分能够在反粗化阶段统一调整多个分区边界，因此通常具有更好的整体平衡性和更低的边界耦合^[13]。

对于本文研究的稀疏 Cholesky 重分解问题， k -way 划分的意义不仅在于减少分区间连接，还在于为后续数值分解构造更平坦的任务结构。通过一次性生成多个相对独立的子图，算法能够形成单层 k 叉树分解结构。该结构具有较浅的树深度和较多的同层独立节点，有助于降低后续 GPU 数值分解中的同步开销，并提供更充足的并行任务粒度。

第三节 嵌套剖分排序算法

ND 是一类经典的稀疏矩阵填充减少排序方法。其基本思想是通过寻找图中的分割顶点集合，将原图递归地分解为若干规模较小的子图，并将分割顶点安排在较晚的消元位置。该方法能够将填充元尽量限制在局部子域内部，从而有效降低 Cholesky 分解因子的非零元数量。

一、顶点分割与分隔集思想

在 ND 中，顶点分割是核心概念。给定图 $G = (V, E)$ ，若存在一个顶点集合 $S \subset V$ ，使得删除 S 后，剩余顶点集合 $V \setminus S$ 可以被划分为两个互不连通的子集 V_1 和 V_2 ，则称 S 为一个顶点分隔集。这里， S 表示分隔两个子图的顶点集合， V_1 和 V_2 分别表示被分隔出来的两个子图顶点集合。

在稀疏矩阵排序中，分隔集对应于连接不同子域的变量。如果过早消去这些变量，可能会在原本相互独立的子域之间引入大量填充元。相反，如果先消去各个子域内部变量，再消去分隔集变量，则填充元主要局限在子域内部，并减少跨子域耦合。因此，ND 通常采用“先子域、后分隔集”的后序排序策略。

二、递归二分排序流程

给定图 $G = (V, E)$ ，ND 通常按照递归二分的方式构造排序。其基本流程如下：

1. 若当前顶点集合 V 的规模小于预设阈值，则停止递归，并对当前子图使用基准排序方法；
2. 否则，在当前图中寻找一个顶点分隔集 S ，使得删除 S 后，剩余图被划分为两个规模相对均衡且互不连通的子图 $G[V_1]$ 和 $G[V_2]$ ；
3. 分别对两个诱导子图 $G[V_1]$ 和 $G[V_2]$ 递归执行 ND，得到两个局部排序 p_1 和 p_2 ；
4. 将两个子图排序与分隔集排序进行拼接，得到当前层的排序

$$p = [p_1, p_2, S], \quad (2.3)$$

其中， p 表示当前层返回的排序序列， p_1 和 p_2 分别表示两个子图的局部排序序列；

5. 递归返回到顶层后，得到全局变量排序。

该排序方式的本质是将图结构转换为一棵层次二叉分解树。树的叶节点对应较小的局部子图，内部节点对应不同层级的分隔集。由于分隔集被安排在较晚的消元位置，子图内部的计算可以在一定程度上独立进行，因此 ND 不仅具有较好的填充减少效果，也具有一定的并行计算潜力。

三、层次二叉结构的局限性

尽管 ND 在稀疏矩阵直接求解中具有重要地位，但其递归二叉结构也存在一定局限。首先，递归二分会产生较深的树结构。每一层划分依赖上一层划分结果，导致算法天然具有逐层推进的特征。其次，树的高层节点数量较少，在并行计算中难以提供足够多的独立任务。再次，不同子图规模可能随递归过程逐渐变得不均衡，进而造成任务负载差异。

对于 CPU 而言，递归二叉树结构可以通过复杂控制流和任务调度机制进行

处理。然而，对于 GPU 后续数值分解而言，过深的树结构和较强的层间依赖可能导致同步频繁、任务粒度不均和线程利用率不足。本文采用单层 k 叉树分解思想，正是为了缓解传统 ND 中层次二叉结构带来的这些问题。

第四节 Parth 稀疏 Cholesky 重分解方法

Parth 稀疏 Cholesky 重分解方法是一种面向动态稀疏结构的增量重分解方法。其主要目标是在矩阵非零模式发生局部变化时，尽可能复用上一时刻的填充减少排序和符号分析结果，仅对受影响的局部区域执行必要的结构更新，从而降低连续求解过程中的符号分析成本和整体分解时间^[12]。

Parth 方法将稀疏矩阵的非零模式表示为图结构。对于动态仿真、局部几何编辑或局部重网格化等应用，矩阵结构变化在图上通常表现为局部边或顶点的增删。为了避免每次变化后都进行全局重新排序，Parth 借助 ND 产生的层次分解树组织矩阵结构，并在局部结构改变时定位变化可能传播的区域。

Parth 方法的基本流程可以概括如下：

1. 在初始化阶段，对原始稀疏矩阵对应的图执行 ND 排序，得到初始置换向量和层次二叉分解树；
2. 当检测到稀疏结构发生变化时，在分解树上标记受影响节点，并确定结构变化向上或向下传播的范围；
3. 对受影响的子树执行必要的重排序和符号更新；
4. 对未受影响的子树，直接复用已有排序、符号结构和相关分解信息；
5. 通过局部更新与结构复用，避免对整个矩阵重新执行完整符号分析。

Parth 方法的重要意义在于，它强调了动态稀疏结构下的局部性利用。对于局部修改而言，若能够将影响范围限制在分解树的局部子树内，则可以显著降低重分解成本。然而，由于 Parth 依赖 ND 生成的层次二叉分解树，其结构仍然具有较强的递归层级特征。对于 GPU 后续数值分解而言，深层二叉树可能导致任务展开不足和同步开销增加。因此，本文在借鉴其局部增量更新思想的基础上，进一步采用单层 k 叉树结构，以提高同层子任务之间的独立性和并行调度潜力。

第五节 消元树

消元树是稀疏矩阵分解中的核心图论数据结构之一。它描述了 Cholesky 分解过程中不同列之间的数据依赖关系，是符号分析、内存预分配和并行调度的重要理论基础^[15]。

一、消元树的定义

给定一个 $n \times n$ 的稀疏对称正定矩阵 A ，其中 n 表示矩阵维度。经过填充减少排序后，矩阵的 Cholesky 分解因子记为 L 。消元树 T 是一个以矩阵列为节点的树状结构，每一个节点对应矩阵的一列。

在消元树中，节点 j 的父节点 $p(j)$ 定义为因子矩阵 L 的第 j 列中，对角线以下第一个非零元素所在的行索引：

$$p(j) = \min\{i > j \mid L_{ij} \neq 0\}. \quad (2.4)$$

其中， j 表示当前列索引， i 表示位于 j 下方的行索引， L_{ij} 表示因子矩阵 L 中第 i 行第 j 列的元素。若第 j 列对角线以下不存在非零元素，则节点 j 没有父节点，并成为消元树或消元森林中的根节点。

二、数据依赖与并行性分析

消元树最直接的作用是刻画稀疏 Cholesky 分解中的数据依赖关系。在数值分解过程中，一个列的更新结果会沿着消元树向其祖先节点传播。若两个节点之间不存在祖先与后代关系，则它们对应的列之间不存在直接依赖，可以在一定条件下并行计算。

因此，消元树为稀疏矩阵分解的并行调度提供了理论依据。不同分支上的列可以被分配给不同计算单元独立执行，而靠近根节点的列通常对应较高层级的汇合计算。这一性质使得消元树不仅是符号分析工具，也是并行分解任务组织的重要结构。

三、符号分解与内存预分配

消元树还可用于预测因子矩阵 L 的非零结构。符号分解阶段不执行实际浮点运算，而是根据原矩阵非零模式、排序结果和消元树结构，推断分解因子中可能出现的非零元位置。

通过符号分解，求解器可以提前确定每一列的非零元数量，并据此完成内存

预分配。这对于稀疏直接求解器十分重要，因为若在数值分解过程中动态分配或扩展存储空间，将显著增加运行时开销并破坏数据访问的连续性。

四、消元树与填充减少排序的关系

填充减少排序算法不仅影响因子矩阵 L 的非零元数量，也会改变消元树的形态。AMD 通常通过局部贪心策略减少局部填充，而 ND 则通过图分割构造更具层次性的消元结构。

在 ND 排序下，子域内部变量通常位于排序前部，分隔集变量位于排序后部。这使得消元树底部包含较多局部子域分支，而靠近根部的节点对应较高层级的分隔变量。该结构在一定程度上具有并行潜力，但其递归二叉形态也可能引入较深的层级依赖。

本文后续提出的单层 k 叉树结构，可以看作是在保留图划分降低耦合思想的基础上，对传统层次二叉消元结构进行更适合 GPU 数值分解的组织方式调整。通过增加同层独立子图数量并降低树深度，算法能够为后续数值分解提供更充分的并行任务来源。

第三章 面向 GPU 数值分解的单层 k 叉增量重排序算法

Parth 方法通过 ND 构建层次二叉分解树，并在稀疏结构发生局部变化时对受影响子树进行增量更新。这种方法能够有效复用历史符号结构，但其分解树具有较深的递归层级，且不同层级之间存在较强的祖先依赖关系。在 CPU 环境下，递归式树遍历、局部子树更新和不规则控制流可以由复杂调度机制较好地处理；然而，在面向 GPU 后续数值分解时，深层二叉结构会带来若干限制。首先，二叉树高层节点数量较少，难以在分解初期提供足够多的并行任务；其次，不同子树规模可能差异较大，容易造成线程负载不均衡；最后，层级递归结构会增加同步和调度开销，不利于 GPU 对规则、大批量任务的高吞吐执行。

针对上述问题，本文算法采用单层 k 叉分解结构替代 Parth 中基于 ND 的深层二叉分解结构。本文并不试图将填充减少排序过程本身完全迁移至 GPU，而是在 CPU 端通过 k -way 图划分构造更适合后续 GPU 数值分解的任务组织形式。与 Parth 的层次二叉树相比，单层 k 叉结构具有更浅的依赖层级、更多同层独立子图以及更直接的分区索引关系。矩阵发生局部修改后，算法可以通过顶点所属子图、跨分区边端点标记以及局部影响传播规则，快速定位受影响的少数子图，并将重排序范围限制在局部区域内。由此得到的分解结构更有利于后续 GPU 数值分解阶段进行批量任务调度、并行局部更新和负载均衡。

本文主要考虑自由度 (Degree of Freedom, DOF) 集合保持不变且图中新增边的更新情形。这一设定对应于局部耦合关系增强、局部约束增加或局部拓扑连接发生变化等常见场景。对于删除边或删除自由度的情形，由于原有填充结构通常仍可作为保守的可行结构，算法可以直接复用已有置换向量；对于新增自由度的情形，可将新增自由度及其关联边涉及的子图标记为第一类受影响子图，并纳入本文后续的局部重排序流程。因此，新增边情形能够覆盖本文增量重排序机制中最核心的结构更新逻辑。

第一节 单层 k 叉分解树的结构

与 ND 所采用的层次递归二叉树不同，本文采用单层 k 叉图划分结构。该结构中，每一个叶节点对应一个划分得到的子图，根节点仅用于表示全局划分关系，并不对应具有实际物理含义的分隔区域。由于后续增量更新过程主要关注顶

点的分区归属、跨分区边端点以及局部扰动的传播范围，因此算法无需显式存储每个子图的完整顶点集合，而是通过若干轻量级映射和标记数组完成结构维护。

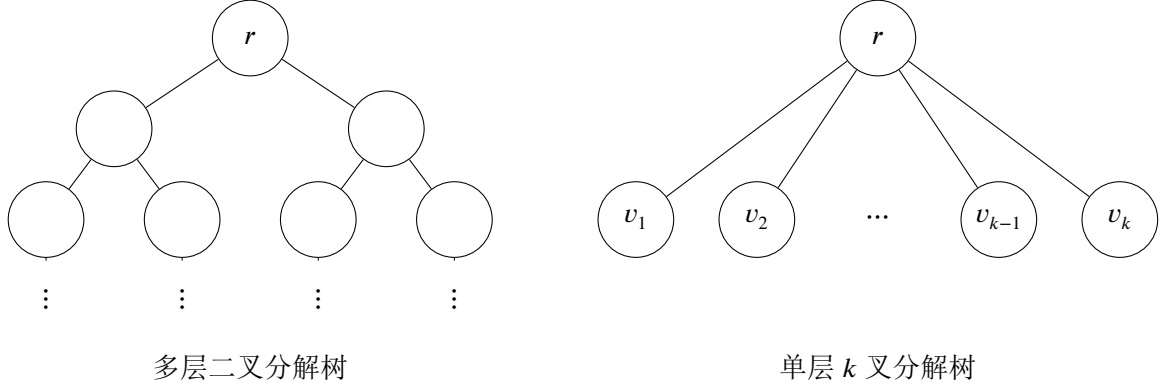


图 3.1 多层二叉分解树与单层 k 叉分解树结构对比

图 3.1 给出了两类分解树结构的示意对比。左侧为 Parth 方法中基于 ND 的多层二叉分解树，其结构递归层级较深；右侧为本文采用的单层 k 叉分解树，其中根节点仅表示全局划分关系，每个叶节点对应一个划分得到的子图。相比多层二叉结构，单层 k 叉结构具有更浅的依赖层级和更多同层独立子图，更适合后续 GPU 数值分解阶段的批量任务调度。

具体而言，本文维护如下三类核心数据。

- k : 整型变量，表示图划分得到的子区域数量，同时也是单层分解树的叉数。
- *DOFToNode*: 长度为 $|V|$ 的整型数组，其中 V 表示图的顶点集合， $|V|$ 表示顶点数量。该数组用于记录每个全局 DOF 所属的子图编号。给定任意顶点，算法可以在常数时间内查询其分区归属，从而避免对子图顶点集合进行显式维护。
- *IsCutVertex*: 长度为 $|V|$ 的布尔数组，用于标记某一顶点是否为跨分区边的端点。若某顶点至少存在一个邻接顶点位于不同子图，则该顶点被标记为切割顶点。该标记用于在新增边后快速判断扰动是否可能跨越当前子图边界传播。

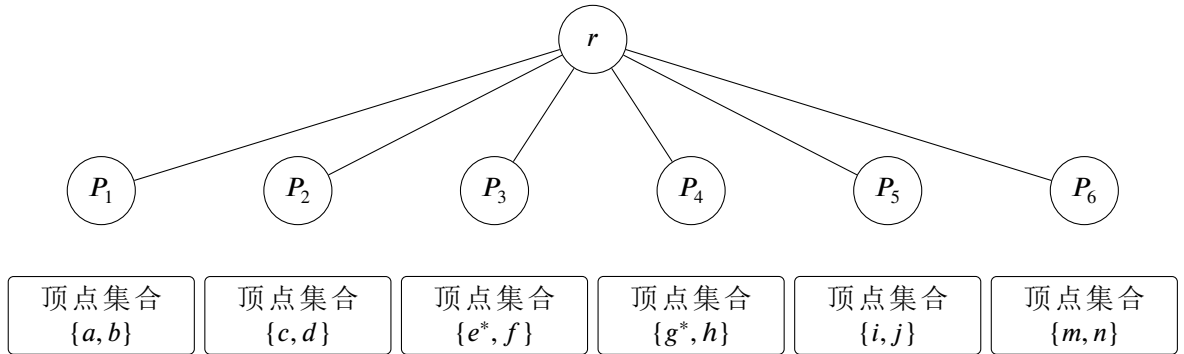
上述数据结构的设计目标是降低增量更新阶段的结构维护成本。通过 *DOFToNode* 数组，算法能够快速确定新增边两端点所在的子图；通过 *IsCutVertex* 数组，算法能够判断局部修改是否涉及分区边界。二者共同构成后续分类重排序策略的基础。

第二节 初始化阶段

初始化阶段的目标是建立全局顶点到单层 k 叉分解结构的映射关系，并计算初始填充减少置换。该阶段首先调用 METIS 的 k -way 划分接口，将原始图划分为 k 个规模相对均衡的子图；随后根据划分结果构造 *DOFToNode* 映射，并扫描所有边以确定切割顶点集合；最后对全图执行 AMD，得到初始置换向量。

设输入图为 $G = (V, E)$ ，其中 E 表示边集合。为了避免划分数超过自由度数量，算法首先将划分数修正为 $\min(k, |V|)$ 。在图为空图时，即非零结构中不存在边，AMD 排序不再具有实际意义，此时算法直接返回恒等置换。否则，算法对全图执行 AMD 排序，得到初始置换向量 *perm*。

初始化阶段得到的子图归属关系、切割顶点标记和初始置换向量将在后续新增边场景中被部分复用。为便于说明，图 3.2 给出了 $k = 6$ 时的单层 k 叉结构示例。其中， P_i 表示第 i 个子图， i 表示子图编号， $a, b, c, d, e, f, g, h, i, j, m, n$ 表示图中的顶点，上标 $*$ 表示切割顶点。示例中， e^* 与 g^* 是由初始化阶段已有跨分区边产生的切割顶点，但是否在后续增量更新中触发第一类标记，仍取决于它们是否参与本轮新增边。



初始化结果：*DOFToNode* 记录每个顶点所属子图，*IsCutVertex* 标记已有跨分区边端点，初始置换向量为 $perm = (a, b, c, d, e, f, g, h, i, j, m, n)$

图 3.2 初始化阶段的单层 k 叉结构示例

其流程如算法 3.1 所示。

算法 3.1 初始化阶段的单层 k 叉结构构造**Data:** 图 $G = (V, E)$, 初始划分数 k **Result:** 子图映射 $DOFToNode$, 切割顶点标记 $IsCutVertex$, 初始置换向量 $perm$

```

1  $k \leftarrow \min(k, |V|)$ ;
2 for  $i \leftarrow 0$  to  $|V| - 1$  do
3    $assigned\_DOF[i] \leftarrow i$ ;
4 end
5  $part \leftarrow \text{METIS\_PartGraphKway}(G, k)$ ;
6 for  $i \leftarrow 0$  to  $|V| - 1$  do
7    $g \leftarrow assigned\_DOF[i]$ ;
8    $DOFToNode[g] \leftarrow part[i]$ ;
9 end
10 foreach 顶点  $u \in V$  do
11    $IsCutVertex[u] \leftarrow 0$ ;
12   foreach 顶点  $u$  的邻接顶点  $v$  do
13     if  $DOFToNode[u] \neq DOFToNode[v]$  then
14        $IsCutVertex[u] \leftarrow 1$ ;
15       break;
16     end
17   end
18 end
19 if  $NNZ(G) = 0$  then
20    $perm \leftarrow (0, 1, \dots, |V| - 1)$ ;
21 else
22    $perm \leftarrow \text{AMD\_order}(G)$ ;
23 end
24 return  $(DOFToNode, IsCutVertex, perm)$ ;

```

第三节 增量二次重排序

当图中新增边后，若重新对全图执行划分与排序，将引入不必要的计算开销，并削弱已有分解结构的复用价值。因此，本文采用增量二次重排序策略。该策略首先比较更新前后的图结构，识别新增边集合；随后根据新增边与已有子图边界之间的关系，将受影响子图划分为不同类型；最后针对不同类型的子图采用不同强度的局部更新策略。

设更新前图为 G_{old} ，更新后图为 G_{new} ，新增边集合定义为

$$E^+ = E(G_{new}) \setminus E(G_{old}).$$

其中， E^+ 表示新增边集合， $E(G_{old})$ 表示旧图边集合， $E(G_{new})$ 表示新图边集合。若 E^+ 为空，则说明图结构未发生需要处理的新增连接，算法可直接返回原有结构。

一、受影响子图分类

本文将受新增边影响的子图分为两类。

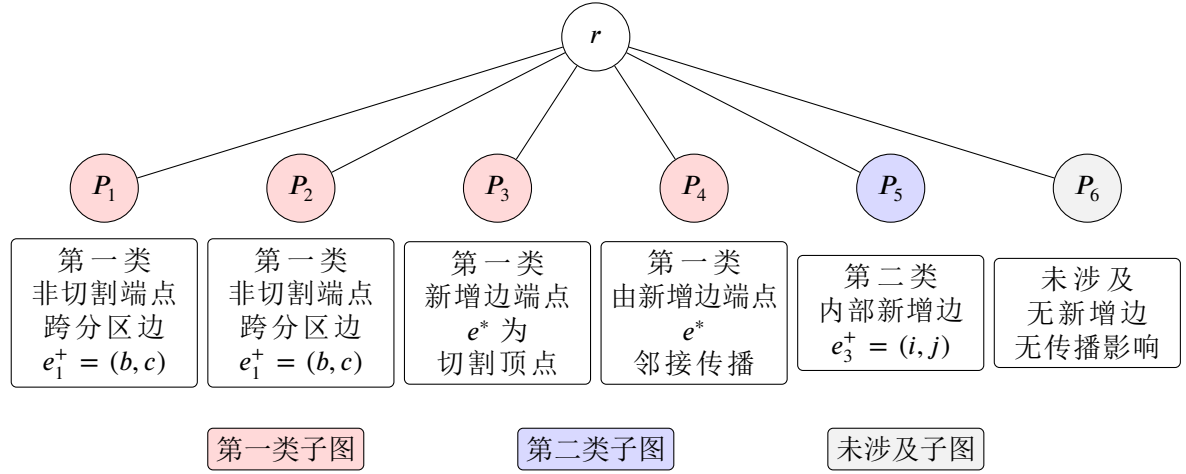
第一类子图表示受到跨分区扰动影响的子图。当新增边的两个端点属于不同子图时，即使这两个端点在更新前均不是切割顶点，该新增边也会引入新的跨分区耦合关系，因此新增边两端点所在子图均被标记为第一类。当新增边的任一端点在更新前已经是切割顶点时，说明本轮新增边发生在已有分区边界附近，该扰动可能通过该切割顶点的已有跨分区连接继续影响相邻子图。因此，算法除标记新增边两端点所在子图外，还会将该切割端点在新图中的邻接顶点所属子图一并标记为第一类。

需要强调的是，切割顶点本身不是独立的触发源。算法只在遍历新增边时检查新增边端点是否为切割顶点。若某个切割顶点没有参与任何新增边，则它不会在本轮更新中单独导致自身所在子图或相邻子图被标记为第一类。

第二类子图表示仅受到子图内部扰动影响的子图。当新增边的两个端点位于同一子图，且两个端点均不是切割顶点时，该新增边的影响可视为局部内部变化。此时仅将该子图标记为第二类，并在后续阶段对其执行轻量级局部排序更新。

第一类标记具有更高优先级。若某一子图同时满足第一类和第二类条件，则最终将其视为第一类子图处理。

图 3.3 给出了子图分类阶段的一个示例。新增边分别记为 e_1^+, e_2^+, e_3^+ 。其中, $e_1^+ = (b, c)$ 表示由非切割顶点形成的跨分区新增边, 因此 P_1 与 P_2 被标记为第一类; $e_2^+ = (e, f)$ 的端点 e^* 是切割顶点, 因此 P_3 被标记为第一类, 并进一步将与 e^* 相邻的顶点所属子图 P_4 标记为第一类; $e_3^+ = (i, j)$ 是普通子图内部新增边, 因此 P_5 被标记为第二类; P_6 未被任何新增边或切割端点传播过程涉及, 因此保持未标记状态。



分类结果: P_1 与 P_2 由非切割端点跨分区新增边触发第一类标记; P_3 由参与新增边的切割顶点 e^* 触发第一类标记; P_4 由该切割端点的邻接传播得到第一类标记; P_5 仅发生普通内部新增边, 因此标记为第二类; P_6 未被新增边或传播过程涉及, 因此保持未标记。

图 3.3 增量二次重排序中的子图分类示例

受影响子图分类过程如算法 3.2 所示。

算法 3.2 增量二次重排序中的受影响子图分类

Data: 旧图 G_{old} , 新图 G_{new} , 子图映射 $DOFToNode$, 切割顶点标记

$IsCutVertex$

Result: 子图影响标记 $mark$

```

1   $E^+ \leftarrow E(G_{\text{new}}) \setminus E(G_{\text{old}})$ ;
2  if  $E^+ = \emptyset$  then
3      return;
4  end
5  对每个分区  $p$ , 令  $mark[p] \leftarrow 0$ ;
6  foreach  $(u, v) \in E^+$  do
7       $p_u \leftarrow DOFToNode[u]$ ;
8       $p_v \leftarrow DOFToNode[v]$ ;
9      if  $p_u \neq p_v$  or  $IsCutVertex[u] = 1$  or  $IsCutVertex[v] = 1$  then
10          $mark[p_u] \leftarrow 1$ ;
11          $mark[p_v] \leftarrow 1$ ;
12         if  $IsCutVertex[u] = 1$  then
13             foreach 顶点  $u$  在  $G_{\text{new}}$  中的邻接顶点  $w$  do
14                  $mark[DOFToNode[w]] \leftarrow 1$ ;
15             end
16         end
17         if  $IsCutVertex[v] = 1$  then
18             foreach 顶点  $v$  在  $G_{\text{new}}$  中的邻接顶点  $w$  do
19                  $mark[DOFToNode[w]] \leftarrow 1$ ;
20             end
21         end
22     else
23         if  $mark[p_u] \neq 1$  then
24              $mark[p_u] \leftarrow 2$ ;
25         end
26     end
27 end

```

二、局部划分与置换更新

在完成子图分类后，算法分别处理第一类子图和第二类子图。

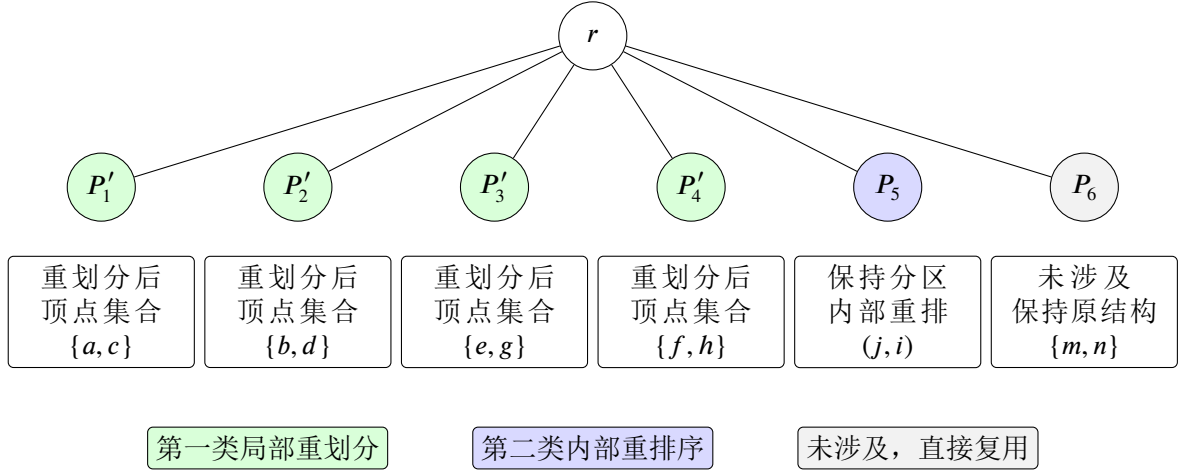
对于第一类子图，算法采用相对较强的局部重构策略。首先，将所有第一类子图的顶点合并，形成顶点集合 V_1 ，并在新图 G_{new} 上提取其诱导子图 G_1 。若第一类子图数量大于一，且诱导子图规模足够大，则对 G_1 再次调用 METIS 的 k-way 划分接口，划分数设为第一类子图的数量。新的划分结果被回填到 *DOFToNode* 中，并尽量沿用原第一类子图的编号空间，以保持全局分区编号的一致性。随后，算法在 G_1 上执行 AMD，得到局部置换，并仅更新这些顶点在全局置换向量中的相对顺序。

对于第二类子图，算法采用轻量级内部调整策略。对于每一个被标记为第二类的子图，算法独立提取其诱导子图，并在该子图内部执行 AMD 排序。该过程只改变子图内部顶点的相对顺序，不改变该子图与其他子图之间的全局排列关系。因此，第二类更新能够以较低代价处理纯局部扰动。

对于未被标记的子图，算法不重新划分，也不重新排序。该类子图既没有直接参与新增边，也没有被切割端点传播过程覆盖，因此可以完全复用原有分区关系和局部置换顺序。

最后，由于第一类子图在局部重划分后可能改变顶点所属分区，算法需要基于更新后的 *DOFToNode* 在新图上重新扫描所有边，并重建 *IsCutVertex* 标记。该步骤保证后续增量更新仍然能够准确判断跨分区扰动范围。

图 3.4 展示了图 3.3 所对应分类结果的一种局部置换更新方式。第一类子图 P_1, P_2, P_3, P_4 被合并为局部更新区域并重新划分；第二类子图 P_5 保持原分区关系，仅在内部更新顶点相对顺序；未被涉及子图 P_6 完全复用原有结构。



置换更新结果：第一类区域对应顶点集合 $V_1 = \{a, b, c, d, e, f, g, h\}$ ，在该区域内重新执行局部划分与 AMD 排序；第二类子图 P_5 不参与跨分区重划分，只对 $\{i, j\}$ 执行内部 AMD 排序；未涉及子图 P_6 保持原有分区和顺序。一个可能的更新后置换向量为 $perm' = (a, c, b, d, e, g, f, h, j, i, m, n)$ 。

图 3.4 增量二次重排序中的局部划分与置换更新示例

局部划分与置换更新过程如算法 3.3 所示。

算法 3.3 增量二次重排序中的局部划分与置换更新

Data: 新图 G_{new} , 子图映射 $DOFToNode$, 切割顶点标记 $IsCutVertex$,
置换向量 $perm$, 子图影响标记 $mark$

Result: 更新后的 $DOFToNode$, $IsCutVertex$, $perm$

```

1   $P_1 \leftarrow \{ p : mark[p] = 1 \};$ 
2   $V_1 \leftarrow \{ u : DOFToNode[u] \in P_1 \};$ 
3   $G_1 \leftarrow G_{\text{new}}[V_1];$ 
4  if  $|P_1| > 1$  and  $|V_1|$  足够大 then
5       $part_1 \leftarrow \text{METIS\_PartGraphKway}(G_1, |P_1|);$ 
6      将  $part_1$  按照  $P_1$  的原编号空间回填至  $DOFToNode$ ;
7  end
8   $perm_1 \leftarrow \text{AMD\_order}(G_1);$ 
9  根据  $perm_1$  更新  $V_1$  中顶点在  $perm$  内的相对顺序;
10 foreach 满足  $mark[p] = 2$  的分区  $p$  do
11      $V_p \leftarrow \{ u : DOFToNode[u] = p \};$ 
12      $G_p \leftarrow G_{\text{new}}[V_p];$ 
13      $perm_p \leftarrow \text{AMD\_order}(G_p);$ 
14     根据  $perm_p$  更新  $V_p$  中顶点在  $perm$  内的相对顺序;
15 end
16 foreach 满足  $mark[p] = 0$  的分区  $p$  do
17     保持该分区的  $DOFToNode$  映射与  $perm$  中的相对顺序不变;
18 end
19 foreach 顶点  $u \in V$  do
20      $IsCutVertex[u] \leftarrow 0;$ 
21     foreach 顶点  $u$  在  $G_{\text{new}}$  中的邻接顶点  $v$  do
22         if  $DOFToNode[u] \neq DOFToNode[v]$  then
23              $IsCutVertex[u] \leftarrow 1;$ 
24             break;
25         end
26     end
27 end
28 return ( $DOFToNode$ ,  $IsCutVertex$ ,  $perm$ );

```

第四节 算法特性分析

本文提出的单层 k 叉增量重排序方法具有以下特点。

首先，该方法将 Parth 中层次二叉分解树的深层递归依赖转化为单层多路划分结构。这样可以增加同层独立子图数量，降低分解树深度，并为后续 GPU 数值分解提供更充足的并行任务来源。

其次，该方法采用数组映射和布尔标记维护子图归属与跨分区边界信息。相比于需要沿层次树结构传播和回溯的更新方式，本文方法在定位受影响区域时具有更直接的访问路径，更适合后续与批量化任务调度相结合。

再次，该方法将局部修改划分为跨分区扰动和子图内部扰动两类。跨分区扰动通过第一类子图处理，必要时重新执行局部 k -way 划分；子图内部扰动通过第二类子图处理，仅执行局部 AMD 调整。该分类机制避免了对全图进行不必要的重新排序。同时，切割顶点只在其参与新增边时触发传播，未参与新增边的已有切割顶点不会单独扩大本轮更新范围，从而避免对受影响区域的过度估计。

最后，该方法在结构复用与局部更新之间取得平衡。未受影响的子图保持原有排序和分区关系，受影响区域则根据扰动类型进行不同强度的更新。这种策略能够在保持分解质量的同时降低局部修改后的重排序开销，从而为大规模网格局部修改场景下的稀疏 Cholesky 重分解提供更高效率的符号结构支持。

第五节 应用场景

本文算法适用于大规模稀疏矩阵仅发生局部结构修改，并且需要反复执行线性系统求解的场景。其主要应用场景包括以下几类。

第一类是交互式几何处理与网格编辑。在局部拖拽、局部变形、约束点增删和局部重网格化过程中，系统矩阵的非零结构通常只在少数区域发生变化。本文算法能够将重排序范围限制在受影响子图内，从而减少每次交互操作后的符号更新开销。

第二类是基于有限元方法的物理仿真。在弹性体仿真、布料仿真、接触响应和局部材料参数变化等问题中，刚度矩阵或约束矩阵可能随时间步发生局部更新。若每一帧都重新执行全局排序和分解，将造成较高的重复计算成本。本文算法通过复用未受影响区域的结构信息，可用于提升连续时间步中的重分解效率。

第三类是动态图形场景中的局部拓扑更新。在碰撞检测、断裂模拟、局部连接关系变化和动态约束引入等情况下，矩阵图结构可能表现为少量边或局部自

由度关系的变化。本文算法针对新增边和局部耦合增强情形设计，因而适合处理这类稀疏且局部的结构扰动。

第四类是面向 GPU 加速的稀疏直接求解预处理。本文算法在 CPU 端完成填充减少排序和分解树构造，并输出更适合 GPU 后续数值分解的单层 k 叉任务结构。因此，该算法可作为 GPU 稀疏 Cholesky 重分解流程中的符号结构准备阶段，为后续并行数值分解、局部更新和任务调度提供结构基础。

综上，本文算法尤其适用于网格规模较大、局部修改频繁、全局重新排序代价较高，并且后续数值计算希望利用 GPU 并行能力加速的应用环境。

第四章 实验评估

第一节 实验环境与测试数据

本文实验在 x86_64 架构平台上完成。硬件平台采用 12th Gen Intel(R) Core(TM) i7-12700H 处理器，该处理器包含 10 个物理核心和 20 个逻辑线程，每个物理核心支持 2 个线程。缓存配置方面，L1d 缓存为 480 KiB，L1i 缓存为 320 KiB，L2 缓存为 12.5 MiB，L3 缓存为 24 MiB。实验机器的内存容量为 7.6 GiB。

软件环境方面，实验运行于 WSL 虚拟化环境下的 Ubuntu 24.04.3 LTS，Linux 内核版本为 6.6.87.2-microsoft-standard-WSL2，编译器采用 GCC 13.3.0。程序使用 C++17 标准实现，构建工具为 CMake 3.28.3。主要依赖库包括 Eigen3 3.4.0、OpenMP 5.1、METIS 5.1.0、SuiteSparse 7.6.1 以及 MATIO 1.5.26。

实验使用三个不同规模的三角网格模型生成 Laplace 矩阵，以评估本文算法在不同问题规模下的运行表现。其中，cathead 网格包含 131 个顶点和 378 条边，ra 网格包含 10044 个顶点和 30132 条边，igea 网格包含 134345 个顶点和 403029 条边。三个网格分别对应小规模、中等规模和大规模测试场景，可用于观察本文算法在不同矩阵规模下的性能变化趋势。

由于 k-way 划分算法具有一定随机性，不同划分结果可能导致运行时间存在波动。为降低随机误差对实验结论的影响，本文对每组参数执行多次测试，并统计其平均运行时间。后续运行时间实验中，cathead、ra 和 igea 在每种分区数下的测试次数分别为 10000 次、1000 次和 1000 次。

第二节 运行时间实验

本节评估本文算法在局部新增边场景下的运行时间表现。实验分别对 cathead、ra 和 igea 三个网格随机新增 2、5、10 条边，并记录首次 k-way 划分、首次 AMD 重排序以及增量二次重排序的平均运行时间。横轴为分区数 k ，其中 k 表示 k-way 划分得到的子区域数量；纵轴为对应阶段的平均运行时间。

一、cathead 网络运行时间分析

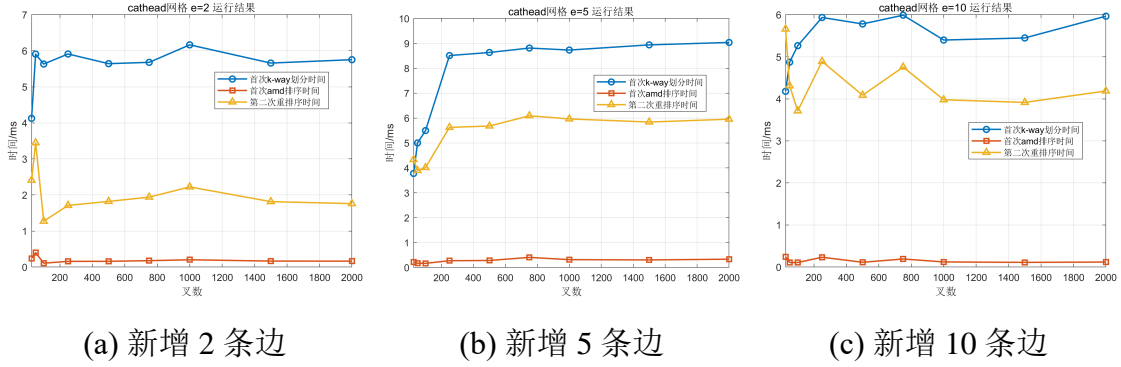


图 4.1 cathead 网络运行时间

图 4.1 展示了小规模 cathead 网络在不同新增边数量下的运行时间变化。由于该网络顶点数和边数较少，直接执行 AMD 重排序的时间开销本身较低，而构造基于分区的层次化结构需要额外调用 k-way 划分并维护分区信息。因此，在小规模问题中，本文方法的初始化开销相对明显，增量二次重排序所带来的收益不足以完全抵消前期划分成本。

这一结果表明，对于规模较小的稀疏矩阵，传统全局 AMD 重排序已经能够在较短时间内完成，本文方法的结构化分区优势尚不显著。本文算法更适用于矩阵规模较大、局部修改频繁且全局重排序成本较高的应用场景。

二、ra 网络运行时间分析

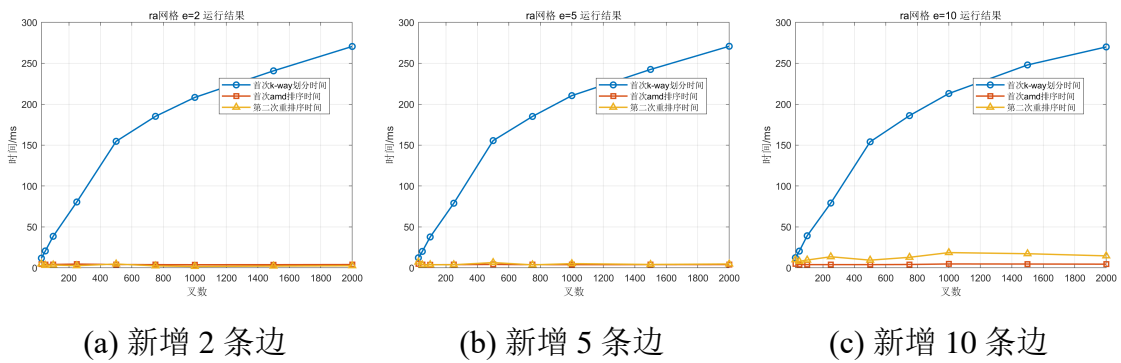


图 4.2 ra 网络运行时间

图 4.2 展示了中等规模 ra 网络上的运行时间结果。与 cathead 相比，ra 网络的顶点数和边数明显增加，直接对全图执行 AMD 重排序的计算开销随之上升。在该规模下，基于分区的局部更新策略开始体现出优势：当新增边仅影响少量局

部区域时，二次重排序可以避免对全图重新排序，从而降低局部修改后的更新成本。

同时可以观察到，随着分区数 k 的增加，初始 k -way 划分成本会相应上升。这是因为更大的 k 会产生更多分区及分区边界，并增加划分结果维护和边界关系处理的复杂度。另一方面，更大的 k 也会使单个子图规模减小，从而降低局部 AMD 调整和二次重排序所涉及的计算范围。因此，ra 网络上的实验结果反映出本文算法中的基本权衡关系：增大分区数有助于缩小局部更新规模，但也会提高初始化阶段的划分成本。

三、igea 网络运行时间分析

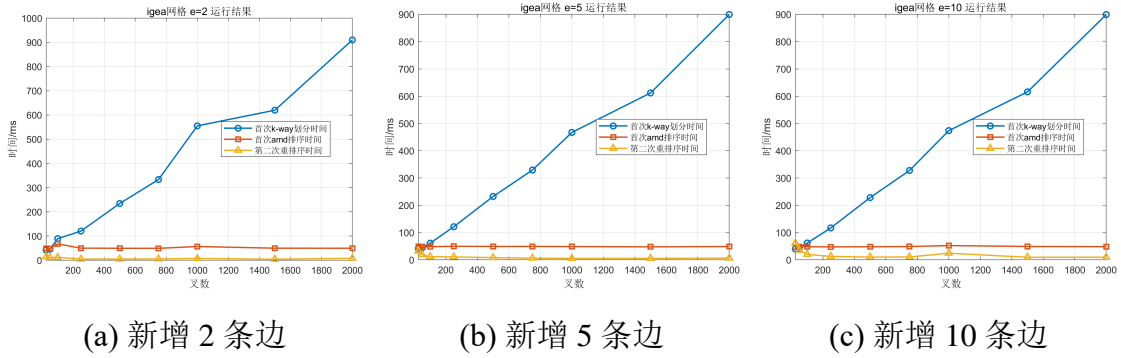


图 4.3 igea 网络运行时间

图 4.3 展示了大规模 igea 网络上的运行时间结果。该网络具有 10^5 量级顶点和 10^5 量级以上边数，直接对全图执行重排序的计算开销较高。在这种规模下，本文算法通过 k -way 划分将全局图拆分为多个相对独立的子图，使新增边引起的结构更新可以被限制在局部区域内，从而更明显地降低二次重排序时间。

与小规模和中等规模网络相比，igea 网络更能体现本文方法的适用场景。随着 k 增大，初始 k -way 划分耗时仍会增加，但局部重排序阶段所需处理的子图规模随之减小。对于局部拓扑更新频繁的大规模网络而言，初始分区结构的构建成本可以在多次增量更新过程中被摊销。因此，在合理范围内增大分区数有助于提升后续连续重分解的整体效率。

第三节 置换结构可视化实验

除运行时间外，本文还通过矩阵非零结构可视化结果评估排序后的结构特征。在理想情况下，填充减少排序应当使置换后的矩阵呈现较明显的分块结构。

本文期望的置换结果可表示为

$$P^T A P = \begin{bmatrix} \Sigma & \alpha \\ \alpha^T & C \end{bmatrix}.$$

其中, P 表示置换矩阵, P^T 表示 P 的转置矩阵, A 表示原始稀疏矩阵, Σ 表示由各个分区内部变量形成的分块对角部分, α 表示分区内部变量与分区边界变量之间的耦合部分, α^T 表示 α 的转置部分, C 表示分区边界变量之间的耦合部分。

下面展示 igea 网格新增 2 条边时的置换结构可视化结果。由于 k-way 划分具有一定随机性, 图中仅展示一次运行中的代表性结果。

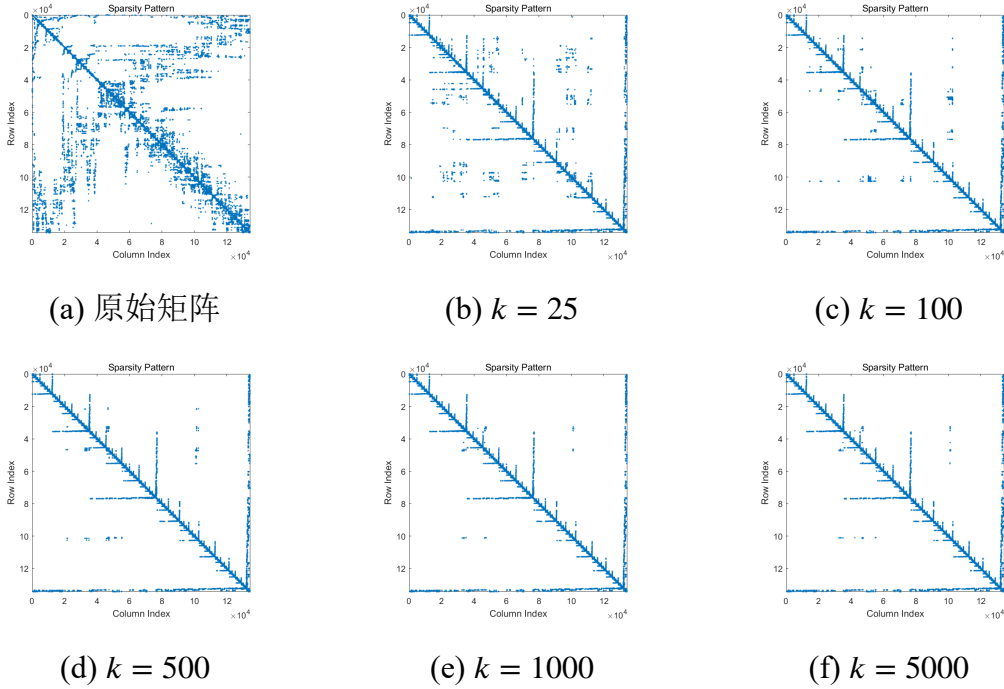


图 4.4 igea 网格置换结构可视化

图 4.4 展示了不同分区数 k 下置换矩阵作用后的非零结构变化。与原始矩阵相比, 经过本文方法排序后的矩阵呈现出更明显的分块特征, 说明 k-way 划分能够将图中强耦合的局部区域组织到相邻的变量区间中。随着 k 增大, 分区数量增加, 单个子图规模减小, 矩阵中的局部分块结构进一步细化。

从重分解角度看, 这种分块结构具有两方面意义。首先, 分块对角区域对应相对独立的子图内部变量, 有利于后续局部数值分解和局部排序更新。其次, 跨分区耦合主要集中在边界相关区域, 使得新增边或局部扰动的影响范围更容易通过切割顶点和子图标记进行定位。因此, 置换结构可视化结果从结构层面验证了本文算法在构造 GPU 友好符号分解结构方面的有效性。

为进一步分析不同置换结构对符号分解依赖关系的影响, 本文在相同实验

设置下绘制置换后的消元树。消元树能够刻画 Cholesky 分解过程中列之间的依赖关系，其高度可用于衡量符号分解中的最长依赖链长度。消元树高度越低，说明消元过程中可并行执行的层次越多，潜在并行性通常越高。

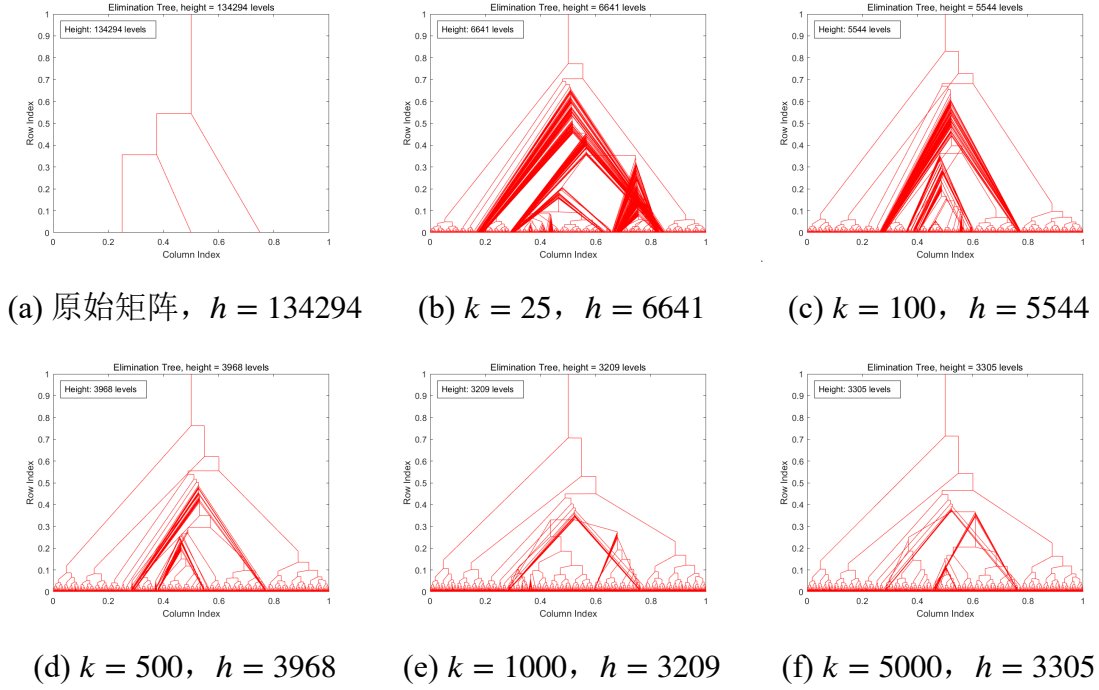


图 4.5 igea 网格置换后的消元树可视化

其中, h 表示消元树高度。图 4.5 中各结构对应的消元树高度如表 4.1 所示。

表 4.1 不同置换结构对应的消元树高度

矩阵结构	消元树高度 h
原始矩阵	134294
$k = 25$	6641
$k = 100$	5544
$k = 500$	3968
$k = 1000$	3209
$k = 5000$	3305

由表 4.1 可见, 原始矩阵对应的消元树高度为 134294, 接近矩阵阶数, 说明原始变量顺序下存在较长的消元依赖链, 符号分解过程具有较强的串行依赖特征。经过本文方法置换后, 消元树高度显著降低。当分区数为 25 时, 消元树高度下降至 6641, 表明置换后的变量组织方式已经明显压缩了消元依赖深度。当分区数进一步增大至 100、500 和 1000 时, 消元树高度分别下降至 5544、3968

和 3209，说明更细粒度的分区能够进一步削弱长距离变量依赖，使消元树结构趋于扁平化。

这一结果与图 4.4 所示的稀疏结构变化具有一致性。置换后的矩阵形成较清晰的分块结构后，子图内部变量被组织到局部连续区域中，跨分区耦合则主要集中在边界相关部分。对于消元树而言，这种结构能够减少由全局变量顺序引起的长链式依赖，使不同分区内部的消元过程具有更好的相对独立性。因此，消元树高度的降低不仅表明符号结构更加紧凑，也说明后续数值分解中潜在的层级并行度得到提升。

需要注意的是，消元树高度并不随分区数增加而严格单调下降。在本组实验中，当分区数为 1000 时，消元树高度为 3209；当分区数继续增大至 5000 时，消元树高度略微上升至 3305。这表明过大的分区数并不必然继续改善消元树结构。其原因在于，分区数增大虽然会降低单个子图的规模，但也会增加分区边界数量和跨分区耦合关系，使边界变量部分可能形成新的依赖集中区域。此外，k-way 划分本身具有一定随机性，过细分区可能放大大局部分区不均衡和边界耦合复杂度对消元树结构的影响。因此，增大分区数可以在一定范围内降低局部子图规模并压缩消元依赖深度，但当分区过细时，边界复杂度的增加可能抵消部分收益。

综合稀疏结构图像和消元树高度结果可以看出，本文构造的置换顺序能够显著降低大规模矩阵的消元依赖深度，并形成更适合并行分解的符号结构。在当前 igea 网格实验中，分区数为 1000 时消元树高度最低，说明该参数在降低依赖链长度与控制边界复杂度之间取得了较好的平衡；而分区数为 5000 时，虽然分区粒度进一步细化，但额外引入的边界耦合开始削弱进一步划分带来的收益。因此，在实际应用中，分区数的选择需要同时考虑子图规模、边界复杂度、初始化划分成本以及后续 GPU 并行分解效率，而不应仅以增大分区数为目标。

第四节 本章小结

本章从运行时间、置换结构和消元树结构三个方面对本文算法进行了实验评估。实验结果表明，对于小规模网格，直接执行 AMD 重排序的时间开销较低，构建基于分区的层次化结构所引入的额外成本难以体现明显优势；对于中等规模网格，本文算法开始表现出局部更新的收益，但仍需要在初始化划分成本和二次重排序效率之间进行权衡；对于 10^5 量级的大规模网格，本文算法能够更有效地限制局部修改的影响范围，降低局部重分解阶段的计算开销。

总体而言，本文方法更适用于网格规模较大、拓扑更新局部且连续重分解次数较多的应用场景。在此类场景中，较大的分区数可以提供更细粒度的子图划分，使局部扰动被限制在更小的区域内；虽然初始 k -way 划分成本会随分区数增大而上升，但该成本可以在多次局部更新中被摊销，从而提高整体计算效率。置换结构可视化结果进一步表明，本文方法能够在保持分解结构质量的同时形成较清晰的分块模式。消元树可视化结果则说明，本文置换结构能够显著降低大规模矩阵的消元树高度，缩短符号分解中的长依赖链，为后续 GPU 数值分解提供更适合并行执行的符号结构基础。

第五章 总结与展望

第一节 工作总结

本文围绕稀疏 Cholesky 重分解中符号分析阶段的高效更新问题，提出并实现了一种面向 SIMT 架构的单层 k 叉图划分与增量重排序方法。该方法以图拓扑的局部变化特征为基础，对传统符号分析流程进行了面向并行计算架构的重构。本文的主要工作可归纳如下：

- **提出单层 k 叉图划分框架，重构稀疏分解拓扑。**针对传统嵌套剖分方法中多级二叉递归结构带来的同步依赖和任务粒度不均衡问题，本文引入基于 METIS 的 k -way 图划分方法，将稀疏矩阵对应的图结构一次性划分为 k 个规模相对均衡的子图。该单层扁平化划分方式弱化了递归层级之间的串行依赖，增加了可并行调度的独立任务数量，从而更适合 GPU 等 SIMT 架构下的大规模并行执行。
- **设计基于局部扰动传播范围的增量重排序机制。**针对动态图仿真和图形计算应用中矩阵拓扑通常仅发生局部修改的特点，本文提出了一种局部化的符号更新策略。通过构建 *DOFToNode* 映射并维护切割顶点标记，算法能够快速定位拓扑变化所影响的子图范围。在此基础上，本文将受扰子图划分为两类：对于跨分区扰动或涉及切割顶点的第一类子图，执行局部统一重分解与重排序；对于扰动仅局限于单个分区内部的第二类子图，仅进行逐分区的局部 AMD 调整。该分类更新机制避免了对全局矩阵重新排序，从而有效降低了增量符号分析阶段的计算开销。
- **通过实验验证算法的有效性与适用性。**本文在 cathead、ra 和 igea 等多组标准测试网络上对所提方法进行了实验评估。运行时间结果表明，增量二次重排序的时间开销显著低于首次全局重排序，说明该方法能够有效利用拓扑变化的局部性。同时，不同分区数下的实验结果表明，分区粒度会影响任务并行度、局部排序开销与边界规模之间的平衡，因此合理选择分区数对于获得较优性能具有重要意义。置换后稀疏结构的可视化结果进一步表明，本文算法能够将原始矩阵转化为具有明显分块对角结构和边界耦合结构的形式，说明该方法在保持填充减少效果的同时，为后续并行数值分解提供了更适合调度的矩阵结构。

第二节 未来展望

尽管本文已在稀疏 Cholesky 重分解的符号分析阶段取得了一定进展，但要将该方法进一步发展为面向实际图形计算和物理仿真任务的高性能求解组件，仍需在以下几个方面继续深入研究：

- **面向 GPU 架构的并行实现与性能优化。**本文当前实验主要验证了单层 k 叉划分与增量重排序方法的算法可行性和局部更新效率。后续工作将进一步面向 CUDA 平台实现局部重分解和多分区局部 AMD 排序过程，并结合 GPU 显存访问特性、线程束级并行机制以及任务批处理策略，对数据布局和执行流程进行系统优化，以充分发挥 GPU 在高吞吐并行计算中的优势。
- **与数值分解阶段的协同设计。**符号分析的最终目标是为数值分解提供高效的计算结构。未来需要结合本文生成的置换向量和分块结构，进一步设计适用于 GPU 的批量 Cholesky 分解核函数与边界耦合处理机制，使符号分析阶段形成的独立子图并行性能够有效传递到数值分解阶段，从而实现从符号分析到数值计算的整体性能提升。
- **扩展对复杂动态拓扑变化的支持。**本文方法主要针对自由度数量保持不变且局部新增边的常见动态更新场景进行优化。后续研究可进一步扩展该框架，使其支持自由度增加、自由度删除、边删除以及大规模拓扑重构等更复杂的变化形式。同时，还需完善扰动影响范围判定和子图分类更新策略，以提高算法在更一般动态稀疏线性系统中的适用性和鲁棒性。

参 考 文 献

- [1] Herholz P, Alexa M. Factor once: reusing cholesky factorizations on sub-meshes [J]. ACM Trans. Graph., 2018, 37(6).
- [2] Herholz P, Sorkine-Hornung O. Sparse cholesky updates for interactive mesh parameterization[J]. ACM Trans. Graph., 2020, 39(6).
- [3] Li J, Liu T, Kavan L, et al. Interactive cutting and tearing in projective dynamics with progressive cholesky updates[J]. ACM Trans. Graph., 2021, 40(6).
- [4] Chan R H, Greif C, O' Leary D P. Methods for modifying matrix factorizations (With P. E. gill, W. murray and M. A. Saunders)[M]//Milestones In Matrix Computation: Selected Works Of Gene H. Golub, With Commentaries. Oxford University Press, 2007.
- [5] Bartels R, Kaufman L. Cholesky factor updating techniques for rank 2 matrix modifications[J]. SIAM Journal on Matrix Analysis and Applications, 1989, 10 (4): 557-592.
- [6] Davis T A, Hager W W. Modifying a sparse cholesky factorization[J]. SIAM Journal on Matrix Analysis and Applications, 1999, 20(3): 606-627.
- [7] Davis T A, Hager W W. Dynamic supernodes in sparse cholesky Update/Downdate and triangular solves[J]. ACM Trans. Math. Softw., 2009, 35 (4).
- [8] Karsavuran M O, Ng E G, Peyton B W. GPU accelerated sparse cholesky factorization[C]//SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2024: 703-707.
- [9] Yuan C. Fast sparse matrix reordering on GPU for cholesky based solvers[D]. University of California, Davis, 2024.
- [10] Yannakakis M. Computing the minimum fill-in is NP-complete[J]. SIAM Journal on Algebraic Discrete Methods, 1981, 2(1): 77-79.
- [11] Amestoy P R, Davis T A, Duff I S. Algorithm 837: AMD, an approximate minimum degree ordering algorithm[J]. ACM Trans. Math. Softw., 2004, 30(3): 381-388.

- [12] Zarebavani B, M. Kaufman D, I. W. Levin D, et al. Adaptive algebraic reuse of reordering in cholesky factorizations with dynamic sparsity patterns[J]. ACM Trans. Graph., 2025, 44(4).
- [13] Karypis G, Kumar V. Multilevelk-way partitioning scheme for irregular graphs [J]. Journal of Parallel and Distributed Computing, 1998, 48(1): 96-129.
- [14] Karypis G, Kumar V. A fast and high quality multilevel scheme for partitioning irregular graphs[J]. SIAM Journal on scientific Computing, 1998, 20(1): 359-392.
- [15] Liu J W. A compact row storage scheme for cholesky factors using elimination trees[J]. ACM Trans. Math. Softw., 1986, 12(2): 127-148.